Indian Institute of Technology Jodhpur

Department of Computer Science & Engineering

CSL 7640: Natural Language Understanding

Assignment 2 – Problem 2

Character-Level Name Generation Using RNN Variants

Submitted by: Neha Sharma
Roll No: M24CSE014
Instructor: Dr. Anand Mishra
Date: 25th March 2026

Contents

1 Introduction **2**

2 Approach **2**
2.1 Dataset (Task 0) 2
2.2 Model Architectures (Task 1) 2
2.2.1 Model 1: Vanilla RNN 2
2.2.2 Model 2: Bidirectional LSTM (BLSTM) 2
2.2.3 Model 3: RNN with Causal Attention 3
2.3 Training 3
2.4 Evaluation (Tasks 2 & 3) 4

3 Results **4**
3.1 Model Configuration 4
3.2 Training Loss 5
3.3 Quantitative Evaluation (Task 2) 5
3.4 Representative Generated Samples 6
3.5 Structure & Failure Analysis (Task 3) 6

4 Observations **6**
4.1 Vanilla RNN 6
4.2 BLSTM 7
4.3 RNN with Attention 7
4.4 Key Takeaways 7

5 Files **8**

1 Introduction

The goal of this problem is to build and compare three recurrent neural network architectures for **generating Indian names at the character level**. Given a training set of 1000 Indian names, each model learns the statistical patterns of character sequences and uses that knowledge to generate new, previously unseen names. This is a **sequence generation task** where the model must learn spelling conventions, common name structures, and the first-name–last-name pattern purely from data.

We implement three models from scratch using PyTorch:

1. Vanilla Recurrent Neural Network (RNN)
2. Bidirectional Long Short-Term Memory (BLSTM)
3. RNN with Basic Attention Mechanism

We then evaluate these models quantitatively (novelty and diversity metrics) and qualitatively (realism and failure analysis).

2 Approach

2.1 Dataset (Task 0)

We created a training set of **1000 unique Indian names** by combining common Indian first names (both male and female) with common Indian surnames. The names are stored in `TrainingNames.txt`, one name per line. Examples include: "Aarav Sharma", "Priya Gupta", "Siddharth Menon", "Deepika Chatterjee".

The character vocabulary was built from all unique characters in the dataset, plus three special tokens: `<SOS>` (start of sequence), `<EOS>` (end of sequence), and `<PAD>` (padding for batching). This gives a total vocabulary of **51 characters**.

2.2 Model Architectures (Task 1)

2.2.1 Model 1: Vanilla RNN

This is the simplest model. At each time step, the current character embedding and the previous hidden state are combined through a **tanh nonlinearity** to produce the next hidden state:

$$h_t = \tanh(W_{ih}\, x_t + W_{hh}\, h_{t-1} + b) \tag{1}$$

where x_t is the character embedding at step t and h_{t-1} is the previous hidden state. A linear layer maps h_t to logits over the vocabulary. During generation, we sample characters one at a time and feed each sampled character back as input for the next step (**autoregressive generation**).

2.2.2 Model 2: Bidirectional LSTM (BLSTM)

The BLSTM processes the input sequence in **both forward and backward directions** using two LSTM layers. At each position, the outputs of both directions are concatenated

to form a richer representation. The LSTM cell uses the standard **gating mechanism**:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad \textbf{(forget gate)} \tag{2}$$
$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad \textbf{(input gate)} \tag{3}$$
$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad \text{(candidate)} \tag{4}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \textbf{(cell state)} \tag{5}$$
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad \textbf{(output gate)} \tag{6}$$
$$h_t = o_t \odot \tanh(c_t) \tag{7}$$

For generation, since future characters are not available, we re-encode the entire partial sequence generated so far at each step and use the last position's output to predict the next character.

2.2.3 Model 3: RNN with Causal Attention

This model extends the Vanilla RNN with a **Bahdanau-style additive attention mechanism**. After the RNN produces hidden states, the attention mechanism computes a context vector for each position by attending over all previous positions. Causal masking ensures no look-ahead, which is critical for generation. The attention computation at position t:

$$e_{t,j} = v^T \tanh(W_h\, h_j + W_s\, h_t) \quad \text{for } j \leq t \tag{8}$$
$$\alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_{k=0}^{t} \exp(e_{t,k})} \tag{9}$$
$$c_t = \sum_{j=0}^{t} \alpha_{t,j}\, h_j \tag{10}$$

The context vector c_t is concatenated with the hidden state h_t and passed through a linear layer for prediction. Without causal masking, the model would learn to rely on future characters during training, which **breaks generation completely**.

2.3 Training

All models were trained using **cross-entropy loss** with teacher forcing. The input is the name prepended with `<SOS>` and the target is the name appended with `<EOS>`. Key training details:

- **Optimizer:** Adam (lr = 0.003)
- **Gradient clipping:** max norm of 5.0 (prevents exploding gradients, especially for vanilla RNN)
- **Padding:** sequences padded to equal length within each batch; padded positions ignored in loss
- **Sampling temperature:** 0.8 during generation (lower = more conservative, higher = more random)

2.4 Evaluation (Tasks 2 & 3)

We evaluated each model by generating 100 names and computing:

- **Novelty Rate:** fraction of generated names not present in the training set.
- **Diversity:** number of unique generated names divided by total generated.

For qualitative analysis, we checked whether generated names follow the "Firstname Lastname" structure with correct capitalization, and counted failure modes like repeated characters, missing spaces, and too-short outputs.

3 Results

3.1 Model Configuration

Model	Hidden	Embed	Dropout	Epochs	Parameters
Vanilla RNN	128	32	0.1	50	**28,947**
BLSTM	64×2	32	0.4	30	**58,387**
RNN + Attention	128	32	0.3	50	**68,371**

Table 1: Architecture details. BLSTM uses hidden size 64 per direction (128 total). All use batch size 64 and learning rate 0.003.

Design choices:

- BLSTM uses a **smaller hidden size** (64 per direction) since bidirectionality already doubles the effective capacity.
- BLSTM trains for only **30 epochs with higher dropout** (0.4) to reduce memorization on the small dataset.
- Attention model uses **dropout 0.3** to regularize the attention weights.

3.2 Training Loss

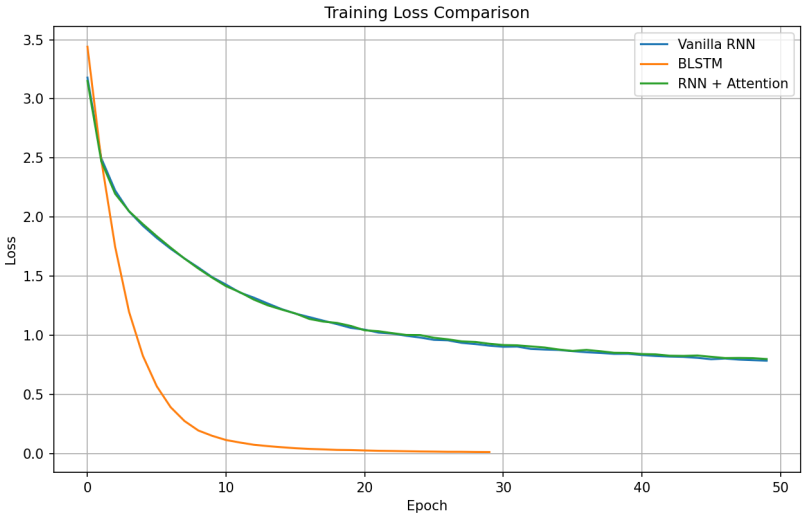

Figure 1: Training loss over epochs for all three models.

Vanilla RNN and Attention RNN converge to similar loss values (∼**0.78–0.80**). BLSTM converges much faster to **near-zero loss** (∼0.01), indicating it memorizes the training data rather than learning generalizable patterns.

3.3 Quantitative Evaluation (Task 2)

Model	Novelty Rate	Diversity	Parameters
Vanilla RNN	0.79	0.99	28,947
BLSTM	**1.00**	**1.00**	58,387
RNN + Attention	0.75	0.99	68,371

Table 2: Quantitative metrics over 100 generated names per model. BLSTM's perfect novelty is misleading – see Observations.

3.4 Representative Generated Samples

Vanilla RNN	BLSTM	RNN + Attention
Rishabh Rathore	Hogopy	Pallavi Nair
Kalpana Kamath	EZdh Sell	Abhishek Arora
Deepika Chatterjee	Zyppedm Jeggul	Urvashi Hooda
Shankar Raval	Dvesh Jedd	Advik Tanwar
Sourav Mehra	Pppasa Mub	Shubham Prasad
Anirudh Dhawan	Krawax Twwra	Darshan Nigam
Ranveer Shah	Uddeep Pall	Juhi Chauhan
Vikram Tiwari	BBhaka Aanwwal	Pankaj Chopra
Dipti Hagderjee	Nicndda YipM	Manveer Yadav
Uma Chaudhary	Gajen Jy	Alpesh Roy

Table 3: 10 representative generated names from each model.

3.5 Structure & Failure Analysis (Task 3)

Model	Well-formed	Single-word	Multi-word	Partial
Vanilla RNN	**50/50**	0	0	0
BLSTM	26/50	10	6	8
RNN + Attention	**50/50**	0	0	0

Table 4: Structural analysis of 50 generated names. "Well-formed" = correct "Firstname Lastname" with proper capitalization.

Failure Type	Vanilla RNN	BLSTM	RNN + Attention
Repeated characters	**0/50**	7/50	**0/50**
Missing space	**0/50**	10/50	**0/50**
Too short (< 3 chars)	**0/50**	2/50	**0/50**

Table 5: Failure mode counts across 50 generated names per model.

4 Observations

4.1 Vanilla RNN

- Produces highly realistic Indian names with **100% well-formed structure** (50/50). Names like "Rishabh Rathore" and "Kalpana Kamath" are indistinguishable from real Indian names.
- Achieves **79% novelty**, meaning about 21% of generated names happen to match training names. This shows the model has learned the underlying distribution well rather than just memorizing.

- Has the **fewest parameters** (28,947) yet performs strongly. The simple autoregressive design aligns naturally with character-by-character generation.
- Occasional minor misspellings occur (e.g., "Hagderjee" instead of "Banerjee"), but these are rare and the names still sound plausibly Indian.

4.2 BLSTM

- This is the **weakest model for generation** despite having the most parameters (58,387). Only **26/50** generated names are well-formed.
- The **100% novelty rate is misleading** – it comes from the generated names being garbled (e.g., "Zyppedm Jeggul", "Pppasa Mub") rather than being creatively novel.
- The core issue is a train-generation mismatch. During training, the backward LSTM sees the **complete sequence including future characters**. During generation, only the partial sequence generated so far is available. The model cannot properly utilize the backward context it learned to depend on.
- Common failures include **repeated characters** ("BBhaka"), **missing spaces**, and nonsensical letter combinations.
- This demonstrates that bidirectional models are better suited for encoding and classification tasks (like sentiment analysis or NER) rather than autoregressive generation.

4.3 RNN with Attention

- Produces the **best overall quality** with **100% well-formed names** (50/50) and **zero detected failures**.
- The causal attention mechanism allows the model to selectively look back at earlier characters when deciding the next character. For example, when generating a surname, the model can attend to the first name to ensure the combination sounds natural.
- Achieves 75% novelty with 99% diversity, indicating it learns generalizable patterns while still being able to reproduce some training names.
- The **causal masking was critical** to making this work. Without it, the attention mechanism creates feedback loops during generation, producing repeated character patterns.

4.4 Key Takeaways

- **Architecture matters more than capacity.** The Attention RNN (68K params) and Vanilla RNN (29K params) both vastly outperform the BLSTM (58K params). The reason is that autoregressive architectures naturally match the generation task, while bidirectional models do not.

- **Regularization is important.** The BLSTM overfits severely (loss ≈ 0.01) even with dropout $= 0.4$ and only 30 epochs. The higher capacity combined with bidirectional context makes it easy for the model to memorize the small training set.
- **Attention helps with consistency.** The Attention RNN produces slightly better names than Vanilla RNN because it can **reference earlier parts of the name**. This is especially helpful for longer names where the vanilla RNN's hidden state may lose information about the first name by the time it generates the surname.
- **Simple models can be surprisingly effective.** The Vanilla RNN, with the fewest parameters and simplest architecture, produces near-perfect names. This suggests that for small-scale sequence generation tasks, a well-tuned simple model can outperform more complex alternatives.

5 Files

- `P2T1.py` – Model definitions, training, and generation
- `P2T2.py` – Quantitative evaluation (novelty, diversity)
- `P2T3.py` – Qualitative analysis (structure, failure modes)
- `generate_names.py` – Training data generation script
- `TrainingNames.txt` – 1000 Indian names (training data)
- `generated_names.txt` – Sample outputs from each model
- `models/` – Saved model weights (`.pt` files)
- `training_loss.png` – Training loss comparison plot